

Bank Management Database System

Project Phase 4 Report: Web Integrations of SQL and NoSQL Databases

Group 10

Borhan Javadian (35012)

Bardiya Shavandi (33588)

Faraz Sahabi (34557)

May 9, 2026

Contents

1	Introduction	3
2	Project Structure	3
3	MySQL Integration	4
4	Trigger Integration	4
4.1	Trigger 1: Account Balance Cannot Go Negative	4
4.2	Trigger 2: Employee Work Date Validation	7
4.3	Trigger 3: Transaction Amount Validation	9
5	Stored Procedure Integration	12
5.1	Procedure 1: Customer Balances in Range	12
5.2	Procedure 2: Employees by Date Range	13
5.3	Procedure 3: Transaction Count by Customer	14
6	MongoDB Support Ticket System	16
6.1	Ticket Listing and Active Username Query	16
6.2	Ticket Creation Query	17
6.3	Ticket Detail Query	18
6.4	Comment Query	19
6.5	Admin Resolve Query	20
7	User Interface Design	23
8	Assumptions and Design Decisions	23
9	Challenges and Fixes	24
10	Conclusion	24
A	AI Usage Documentation	25
A.1	Borhan Javadian	25
A.2	Bardiya Shavandi	25
A.3	Faraz Sahabi	25
B	Integration Notes	26

1 Introduction

This phase extends the Bank Management Database System into a web application. The system exposes the previously implemented MySQL triggers and stored procedures through PHP pages and adds a MongoDB-backed support ticket workflow. The user side allows testing of three triggers, execution of three stored procedures, ticket creation, ticket listing, and ticket commenting. The admin side allows administrators to view all active tickets, add admin comments, and mark tickets as resolved.

The implementation uses PHP with MySQL for the relational database functionality and MongoDB\Driver\Manager for support ticket storage. The project is organized under a `Scripts` directory with separate `user` and `admin` folders.

2 Project Structure

The submitted script folder is organized as follows:

```
Scripts/
|-- admin/
|   |-- common_ui.php
|   |-- detail.php
|   |-- index.php
|   |-- ticket_store.php
|
|-- user/
|   |-- common_ui.php
|   |-- db.php
|   |-- index.php
|   |-- procedure_1.php
|   |-- procedure_2.php
|   |-- procedure_3.php
|   |-- ticket_create.php
|   |-- ticket_detail.php
|   |-- ticket_store.php
|   |-- tickets.php
|   |-- trigger_1.php
|   |-- trigger_2.php
|   |-- trigger_3.php
|
|-- SQLDump.sql
```

The user home page is implemented in `user/index.php`. It provides cards for all three triggers and all three stored procedures, and also links to the MongoDB support ticket system. The admin home page is implemented in `admin/index.php`; it lists all active tickets regardless of username.

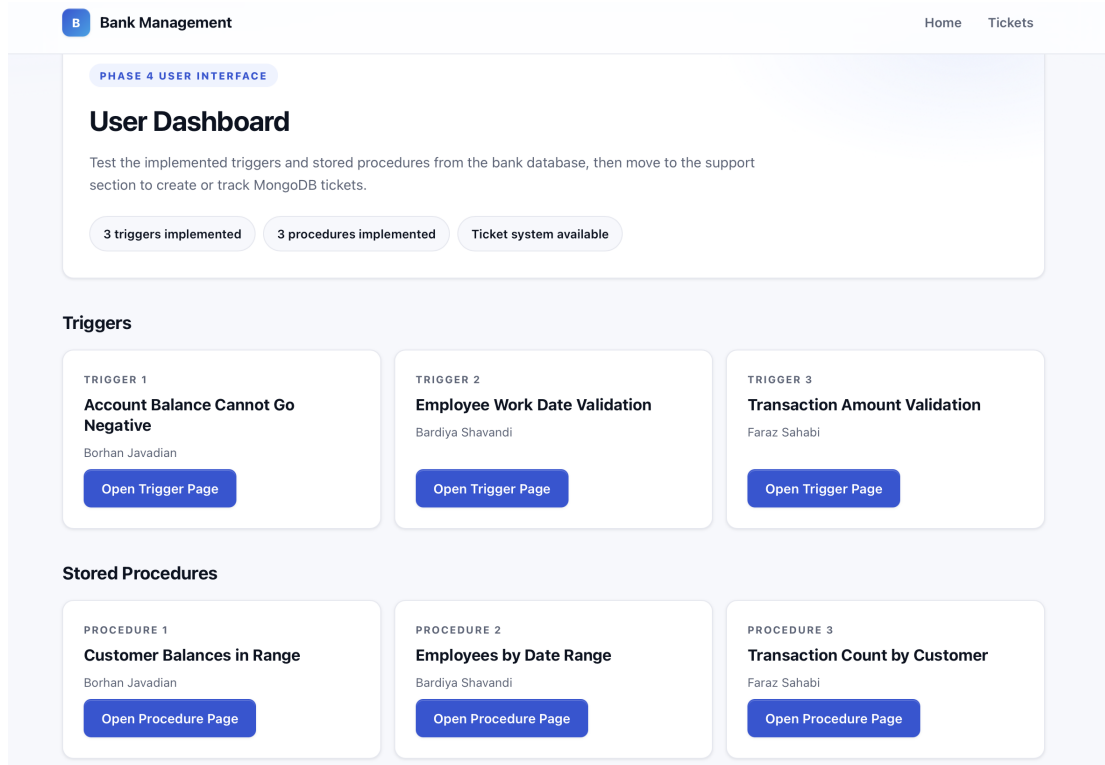


Figure 1: User dashboard showing the trigger, stored procedure, and ticket system navigation.

3 MySQL Integration

The MySQL database is accessed through a shared connection file, `user/db.php`. The project uses `mysqli_report(MYSQLI_REPORT_OFF)` because PHP 8 can surface MySQLi errors as exceptions. Disabling exception mode allowed trigger errors raised with `SIGNAL` to be captured through `$stmt->error` and displayed inside the web page instead of producing blank pages.

Listing 1: Shared MySQL connection used by user-side trigger and procedure pages.

```
mysqli_report(MYSQLI_REPORT_OFF);

$mysqli = new mysqli('localhost', 'root', '', 'bankmanagement');

if ($mysqli->connect_error) {
    die('Database connection failed: ' . $mysqli->connect_error);
}

$mysqli->set_charset('utf8mb4');
```

4 Trigger Integration

4.1 Trigger 1: Account Balance Cannot Go Negative

Responsible member: Borhan Javadian.

Purpose: This trigger prevents an account update from setting the account balance below zero. It fires before an update on the `Account` table. If the new balance is negative, the trigger blocks the update and returns the message `Account balance cannot go negative`.

Listing 2: SQL script for Trigger 1.

```
DELIMITER $$

CREATE TRIGGER trg_prevent_negative_balance
BEFORE UPDATE ON Account
FOR EACH ROW
BEGIN
    IF NEW.Balance < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Account balance cannot go negative';
    END IF;
END$$

DELIMITER ;
```

Web page: `user/trigger_1.php`

Inputs and cases:

- Account dropdown: populated from `SELECT AccountID, Number, Balance FROM Account ORDER BY AccountID`.
- New balance input: submitted to test whether the update is valid.
- Case 1: a non-negative balance, such as 1000, is accepted.
- Case 2: a negative balance, such as -100, is rejected by the trigger.

Listing 3: PHP integration for Trigger 1.

```
$stmt = $mysqli->prepare('UPDATE Account SET Balance = ? WHERE AccountID = ?')
;
$stmt->bind_param('di', $newBalance, $accountId);

if ($stmt->execute()) {
    $messageType = 'success';
    $message = "UPDATE accepted. Account {$accountId} balance was set to {
    $newBalance}.";
} else {
    $messageType = 'error';
    $message = 'UPDATE blocked by trigger: ' . $stmt->error;
}
```

B

Bank Management

Home

Tickets

Trigger 1: Account Balance Cannot Go Negative

RESPONSIBLE: BORHAN JAVADIAN

`trg_prevent_negative_balance` fires *BEFORE UPDATE* on `Account` . If the new `Balance` is negative, the trigger raises a `SIGNAL` and blocks the update. Successful updates persist in the database; the dropdown above is refreshed after each attempt so it always shows the current balance.

Try an Update

ACCOUNT

ACC-002 - current balance: 1000.00

NEW BALANCE

1000

Try Update

TRIGGER ALLOWED THE UPDATE

UPDATE accepted. Account 2 balance was set to 1000.

Suggested cases

- **Allowed:** set the balance to any non-negative value such as `10000` .
- **Blocked:** try a negative value such as `-500` .
- **Allowed:** zero is valid, so `0` should pass.

Figure 2: Trigger 1 successful update with a valid non-negative balance.

B

Bank Management

Home

Tickets

Trigger 1: Account Balance Cannot Go Negative

RESPONSIBLE: BORHAN JAVADIAN

`trg_prevent_negative_balance` fires *BEFORE UPDATE* on `Account` . If the new `Balance` is negative, the trigger raises a `SIGNAL` and blocks the update. Successful updates persist in the database; the dropdown above is refreshed after each attempt so it always shows the current balance.

Try an Update

ACCOUNT

ACC-002 - current balance: 1000.00

NEW BALANCE

-100

Try Update

TRIGGER BLOCKED THE UPDATE

UPDATE blocked by trigger: Account balance cannot go negative

Suggested cases

- **Allowed:** set the balance to any non-negative value such as `10000` .
- **Blocked:** try a negative value such as `-500` .
- **Allowed:** zero is valid, so `0` should pass.

Figure 3: Trigger 1 blocked update when the submitted balance is negative.

4.2 Trigger 2: Employee Work Date Validation

Responsible member: Bardiya Shavandi.

Purpose: This trigger validates inserted rows in the `WorksIn` table. It blocks records where `Until` is before or equal to `StartFrom`. It also blocks records where `StartFrom` is in the future.

Listing 4: SQL script for Trigger 2.

```
DELIMITER $$

CREATE TRIGGER trg_validate_worksin_dates
BEFORE INSERT ON WorksIn
FOR EACH ROW
BEGIN
    IF NEW.Until IS NOT NULL AND NEW.Until <= NEW.StartFrom THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'End date cannot be before or equal to start date';
    END IF;

    IF NEW.StartFrom > CURRENT_DATE THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Start date cannot be in the future';
    END IF;
END$$

DELIMITER ;
```

Web page: `user/trigger_2.php`

Inputs and cases:

- Employee dropdown: populated from the `Employee` table.
- Branch dropdown: populated from the `Branch` table.
- `StartFrom` date input.
- Optional `Until` date input.
- Case 1: valid date interval, accepted and inserted into `WorksIn`.
- Case 2: `Until <= StartFrom`, blocked by the trigger.
- Case 3: future `StartFrom`, blocked by the trigger.

Listing 5: PHP integration for Trigger 2.

```
$stmt = $mysqli->prepare(
    'INSERT INTO WorksIn (EmployeeID, BranchID, StartFrom, Until) VALUES (?,
    ?, ?, ?)'
);
$stmt->bind_param('iiss', $employeeId, $branchId, $startFrom, $untilValue);

if ($stmt->execute()) {
    $messageType = 'success';
}
```

```

$message = "INSERT accepted. Row (EmployeeID={$employeeId}, BranchID={
$branchId}, StartFrom={$startFrom}, Until={$printedUntil}) was added to
WorksIn.";
} else {
$messageType = 'error';
$message = 'INSERT blocked: ' . $stmt->error;
}

```

B

Bank Management

[Home](#)
[Tickets](#)

Trigger 2: Employee Work Date Validation

RESPONSIBLE: BARDIYA SHAVANDI

Try an Insert

EMPLOYEE

2 — Grace Hall

BRANCH

3 — 789 Berliner Str, Berlin

START FROM

12/22/2000

UNTIL (OPTIONAL)

05/07/2026

Successful inserts persist in the WorksIn table; you can verify them in phpMyAdmin. Re-submitting the same Employee + Branch + StartFrom combination replaces the previous row so the trigger can be tested repeatedly.

Try Insert

TRIGGER ALLOWED THE INSERT

INSERT accepted. Row (EmployeeID=2, BranchID=3, StartFrom=2000-12-22, Until=NULL) was added to WorksIn.

Suggested cases

- Allowed: Employee 1 , Branch 2 , StartFrom 2024-03-01 , Until empty.
- Blocked (Until ≤ StartFrom): Employee 2 , Branch 3 , StartFrom 2024-01-01 , Until 2023-06-01 .

Figure 4: Trigger 2 accepted insertion with valid employment dates.

8

Bank Management

[Home](#)
[Tickets](#)

Trigger 2: Employee Work Date Validation

RESPONSIBLE: BARDIYA SHAVANDI

`trg_validate_worksin_dates` fires *BEFORE INSERT* on `WorksIn`. It blocks rows where `Until` is not later than `StartFrom`, and it also blocks rows whose `StartFrom` is in the future.

Try an Insert

EMPLOYEE
2 — Grace Hall

BRANCH
3 — 789 Berliner Str, Berlin

START FROM
12/22/2000

UNTIL (OPTIONAL)
02/02/1999

Successful inserts persist in the `WorksIn` table; you can verify them in phpMyAdmin. Re-submitting the same Employee + Branch + StartFrom combination replaces the previous row so the trigger can be tested repeatedly.

Try Insert

TRIGGER BLOCKED THE INSERT

INSERT blocked: End date cannot be before or equal to start date

Suggested cases

- **Allowed:** Employee `1`, Branch `2`, StartFrom `2024-03-01`, Until empty.
- **Blocked (Until ≤ StartFrom):** Employee `2`, Branch `3`, StartFrom `2024-01-01`, Until `2023-06-01`.

Figure 5: Trigger 2 blocked insertion with invalid employment dates.

4.3 Trigger 3: Transaction Amount Validation

Responsible member: Faraz Sahabi.

Purpose: This trigger prevents inserting transactions with zero or negative amounts. It fires before insert on the `Transaction` table and blocks invalid transaction amounts.

Listing 6: SQL script for Trigger 3.

```

DELIMITER $$

CREATE TRIGGER trg_validate_transaction_amount
BEFORE INSERT ON `Transaction`
FOR EACH ROW
BEGIN
    IF NEW.Amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Transaction amount must be strictly positive';
    END IF;
END$$

DELIMITER ;

```

An additional internal helper trigger was also used to keep account balances synchronized with inserted transactions. It is not shown as a separate homepage feature, but it supports integrated testing because a successful transaction decreases the corresponding account balance.

Listing 7: Internal helper trigger used to apply transaction amounts to balances.

```
DELIMITER $$

CREATE TRIGGER trg_apply_transaction_to_balance
AFTER INSERT ON 'Transaction'
FOR EACH ROW
BEGIN
    UPDATE Account
    SET Balance = Balance - NEW.Amount
    WHERE AccountID = NEW.AccountID;
END$$

DELIMITER ;
```

Web page: user/trigger_3.php

Inputs and cases:

- Account dropdown: populated from the `Account` table.
- Amount input: submitted as the transaction amount.
- Case 1: positive amount within the account balance is accepted.
- Case 2: zero or negative amount is blocked by `trg_validate_transaction_amount`.
- Case 3: positive amount larger than the account balance is blocked indirectly by Trigger 1 after the helper trigger tries to reduce the balance below zero.

Listing 8: PHP integration for Trigger 3.

```
$maxRes = $mysqli->query('SELECT MAX(TransactionID) AS max_id FROM '
    Transaction'');
$nextId = ((int) ($maxRes->fetch_assoc()['max_id'] ?? 0)) + 1;

$stmt = $mysqli->prepare(
    'INSERT INTO 'Transaction' (TransactionID, Amount, TransDate, AccountID)
    VALUES (?, ?, ?, ?)'
);
$stmt->bind_param('idsi', $nextId, $amount, $transDate, $accountId);

if ($stmt->execute()) {
    $messageType = 'success';
    $message = "INSERT accepted. Transaction {$nextId} was recorded and the
    account balance decreased.";
} else {
    $messageType = 'error';
    $message = 'INSERT blocked by trigger: ' . $stmt->error;
}
```

Trigger 3: Transaction Amount Validation

RESPONSIBLE: FARAZ SAHABI

`trg_validate_transaction_amount` fires *BEFORE INSERT* on `Transaction`. If the new `Amount` is zero or negative, the trigger raises a `SIGNAL` and blocks the insert. Successful inserts persist in the `Transaction` table and the linked `Account.Balance` is decremented by the transaction amount automatically (handled in SQL by an internal *AFTER INSERT* trigger). If the resulting balance would go negative, Trigger 1's *BEFORE UPDATE* check fires and the whole insert is rolled back, so neither table changes — you can verify both behaviors in phpMyAdmin.

Try an Insert

ACCOUNT

ACC-001 - balance: 30.00

AMOUNT

5

Try Insert

TRIGGER ALLOWED THE INSERT

INSERT accepted. Transaction 17 (AccountID=1, Amount=5, Date=2026-05-07) was recorded and AccountID=1 balance decreased by 5.

Suggested cases

- **Allowed:** a positive amount that is less than or equal to the account's current balance, such as `50` on an account with `100`.
- **Blocked by Trigger 3:** a negative or zero amount such as `-10` or `0`.
- **Blocked by Trigger 1 (cascade):** a positive amount larger than the account's balance, such as `9999` on an account with `100` — the linked balance update would go negative, so Trigger 1 raises a `SIGNAL` and the entire *INSERT* is rolled back.

Figure 6: Trigger 3 accepted insertion with a positive transaction amount.

Trigger 3: Transaction Amount Validation

RESPONSIBLE: FARAZ SAHABI

`trg_validate_transaction_amount` fires *BEFORE INSERT* on `Transaction`. If the new `Amount` is zero or negative, the trigger raises a `SIGNAL` and blocks the insert. Successful inserts persist in the `Transaction` table and the linked `Account.Balance` is decremented by the transaction amount automatically (handled in SQL by an internal *AFTER INSERT* trigger). If the resulting balance would go negative, Trigger 1's *BEFORE UPDATE* check fires and the whole insert is rolled back, so neither table changes — you can verify both behaviors in phpMyAdmin.

Try an Insert

ACCOUNT

ACC-001 - balance: 25.00

AMOUNT

-9

Try Insert

TRIGGER BLOCKED THE INSERT

INSERT blocked by trigger: Transaction amount must be strictly positive

Suggested cases

- **Allowed:** a positive amount that is less than or equal to the account's current balance, such as `50` on an account with `100`.
- **Blocked by Trigger 3:** a negative or zero amount such as `-10` or `0`.
- **Blocked by Trigger 1 (cascade):** a positive amount larger than the account's balance, such as `9999` on an account with `100` — the linked balance update would go negative, so Trigger 1 raises a `SIGNAL` and the entire *INSERT* is rolled back.

Figure 7: Trigger 3 blocked insertion with a zero or negative amount.

5 Stored Procedure Integration

5.1 Procedure 1: Customer Balances in Range

Responsible member: Borhan Javadian.

Purpose: This procedure returns customers and accounts whose balances are between a minimum and maximum value.

Listing 9: SQL script for Stored Procedure 1.

```
DELIMITER $$

CREATE PROCEDURE GetCustomersInBalanceRange(
    IN p_min DECIMAL(15, 2),
    IN p_max DECIMAL(15, 2)
)
BEGIN
    SELECT C.CustomerID, C.FullName, C.Phone, C.Email,
           A.Number AS AccountNumber, A.Balance
    FROM Customer C
    JOIN Account A ON C.CustomerID = A.CustomerID
    WHERE A.Balance BETWEEN p_min AND p_max
    ORDER BY A.Balance ASC;
END$$

DELIMITER ;
```

Web page: user/procedure.1.php

Input boxes:

- min_balance: minimum balance.
- max_balance: maximum balance.

Listing 10: PHP call for Stored Procedure 1.

```
$stmt = $mysqli->prepare('CALL GetCustomersInBalanceRange(?, ?)');
$stmt->bind_param('dd', $minBalance, $maxBalance);
$stmt->execute();
$result = $stmt->get_result();
```

B

Bank Management

[Home](#)
[Tickets](#)

Procedure 1: Customer Balances in Range

RESPONSIBLE: BORHAN JAVADIAN

GetCustomersInBalanceRange(p_min, p_max) returns every customer-account row whose Balance falls inside the given range, ordered by balance ascending.

Parameters

MINIMUM BALANCE

100

MAXIMUM BALANCE

20000

Call Procedure

Results

3 row(s) returned.

CUSTOMERID	FULLNAME	PHONE	EMAIL	ACCOUNTNUMBER	BALANCE
2	Paula Reed	05302223344	paula@email.com	ACC-002	1000.00
6	Tina Brooks	05306667788	tina@email.com	ACC-006	3200.00
4	Rachel Ford	NULL	rachel@email.com	ACC-004	5000.00

Figure 8: Stored Procedure 1 output for a submitted balance range.

5.2 Procedure 2: Employees by Date Range

Responsible member: Bardiya Shavandi.

Purpose: This procedure returns employees whose WorksIn.StartFrom date is inside the submitted date interval.

Listing 11: SQL script for Stored Procedure 2.

```

DELIMITER $$

CREATE PROCEDURE GetEmployeesByDateRange(
    IN p_start DATE,
    IN p_end DATE
)
BEGIN
    SELECT DISTINCT E.EmployeeID, E.FullName, E.Position, E.Salary,
        W.StartFrom, W.Until, B.Address AS BranchAddress
    FROM Employee E
    JOIN WorksIn W ON E.EmployeeID = W.EmployeeID
    JOIN Branch B ON W.BranchID = B.BranchID
    WHERE W.StartFrom >= p_start
        AND W.StartFrom <= p_end;
END$$

DELIMITER ;

```

Web page: user/procedure.2.php

Input boxes:

- `start_date`: beginning of the date range.
- `end_date`: end of the date range.

Listing 12: PHP call for Stored Procedure 2.

```
$stmt = $mysqli->prepare('CALL GetEmployeesByDateRange(?, ?)');  
$stmt->bind_param('ss', $startDate, $endDate);  
$stmt->execute();  
$result = $stmt->get_result();
```

B

Bank Management

Home Tickets

Procedure 2: Employees by Date Range

RESPONSIBLE: BARDIYA SHAVANDI

`GetEmployeesByDateRange(p_start, p_end)` returns every employee whose `WorksIn` record started inside the supplied date range, together with their branch information. Open-ended assignments whose `StartFrom` falls outside the range are not reported.

Parameters

START DATE

02/07/2022

END DATE

12/02/2025

Call Procedure

Results

4 row(s) returned.

EMPLOYEEID	FULLNAME	POSITION	SALARY	STARTFROM	UNTIL	BRANCHADDRESS
1	Frank White	Teller	35000.00	2024-03-01	NULL	456 High St, London
1	Frank White	Teller	35000.00	2024-05-03	2025-05-02	456 High St, London
1	Frank White	Teller	35000.00	2025-01-01	NULL	456 High St, London
7	Leo King	Advisor	60000.00	2023-01-01	NULL	99 Ataturk Blvd, Ankara

Figure 9: Stored Procedure 2 output for a submitted work date range.

5.3 Procedure 3: Transaction Count by Customer

Responsible member: Faraz Sahabi.

Purpose: This procedure receives a customer ID and returns the number of transactions associated with that customer through their accounts.

Listing 13: SQL script for Stored Procedure 3.

```
DELIMITER $$  
  
CREATE PROCEDURE GetTransactionCountByCustomer(  
    IN p_customer_id INT  
)
```

```

BEGIN
    SELECT COUNT(T.TransactionID) AS TransactionCount
    FROM Account A
    JOIN 'Transaction' T ON A.AccountID = T.AccountID
    WHERE A.CustomerID = p_customer_id;
END$$

DELIMITER ;

```

Web page: user/procedure_3.php

Input element:

- Customer dropdown: populated from `SELECT CustomerID, FullName FROM Customer ORDER BY FullName`.

Listing 14: PHP call for Stored Procedure 3.

```

if ($res = $mysqli->query('SELECT CustomerID, FullName FROM Customer ORDER BY
    FullName')) {
    while ($row = $res->fetch_assoc()) {
        $customers[] = $row;
    }
}

$stmt = $mysqli->prepare('CALL GetTransactionCountByCustomer(?)');
$stmt->bind_param('i', $customerId);
$stmt->execute();
$result = $stmt->get_result();

```

B

Bank Management

[Home](#)
[Tickets](#)

Procedure 3: Transaction Count by Customer

RESPONSIBLE: FARAZ SAHABI

`GetTransactionCountByCustomer(p_customer_id)` calculates and returns the total number of transactions for the specified customer across all their accounts.

Run Procedure

CUSTOMER

Paula Reed

⌵

Execute

Result

CUSTOMER

Paula Reed

CustomerID 2

TOTAL TRANSACTIONS

1

Across all of the customer's accounts

[← Back to dashboard](#)

Figure 10: Stored Procedure 3 output showing transaction count for a selected customer.

6 MongoDB Support Ticket System

The support ticket system stores ticket documents in MongoDB using the namespace `tickets.entries`. Each ticket document contains a username, body, active/resolved status, creation timestamp, and an array of comments.

Listing 15: MongoDB connection constants and manager.

```
const TICKETS_DB = 'tickets';
const TICKETS_COLLECTION = 'entries';
const TICKETS_NAMESPACE = 'tickets.entries';

function ticket_manager()
{
    static $manager = null;

    if ($manager === null) {
        $manager = new MongoDB\Driver\Manager('mongodb://localhost:27017');
    }

    return $manager;
}
```

6.1 Ticket Listing and Active Username Query

The ticket list page uses a MongoDB `distinct` command to retrieve usernames that currently have active tickets. This creates a lightweight user filter for the ticket list page.

Listing 16: MongoDB query for active usernames.

```
$command = new MongoDB\Driver\Command([
    'distinct' => TICKETS_COLLECTION,
    'key' => 'username',
    'query' => ['status' => true],
]);

$result = ticket_manager()->executeCommand(TICKETS_DB, $command)->toArray();
$usernames = isset($result[0]->values) ? (array) $result[0]->values : [];
natcasesort($usernames);
```

Active tickets are queried with `status => true`. When a username is selected, the username is added to the filter.

Listing 17: MongoDB query for active tickets, optionally filtered by username.

```
$filter = ['status' => true];

if ($username !== null && $username !== '') {
    $filter['username'] = $username;
}

$query = new MongoDB\Driver\Query($filter, ['sort' => ['created_at' => -1]]);
return ticket_manager()->executeQuery(TICKETS_NAMESPACE, $query)->toArray();
```

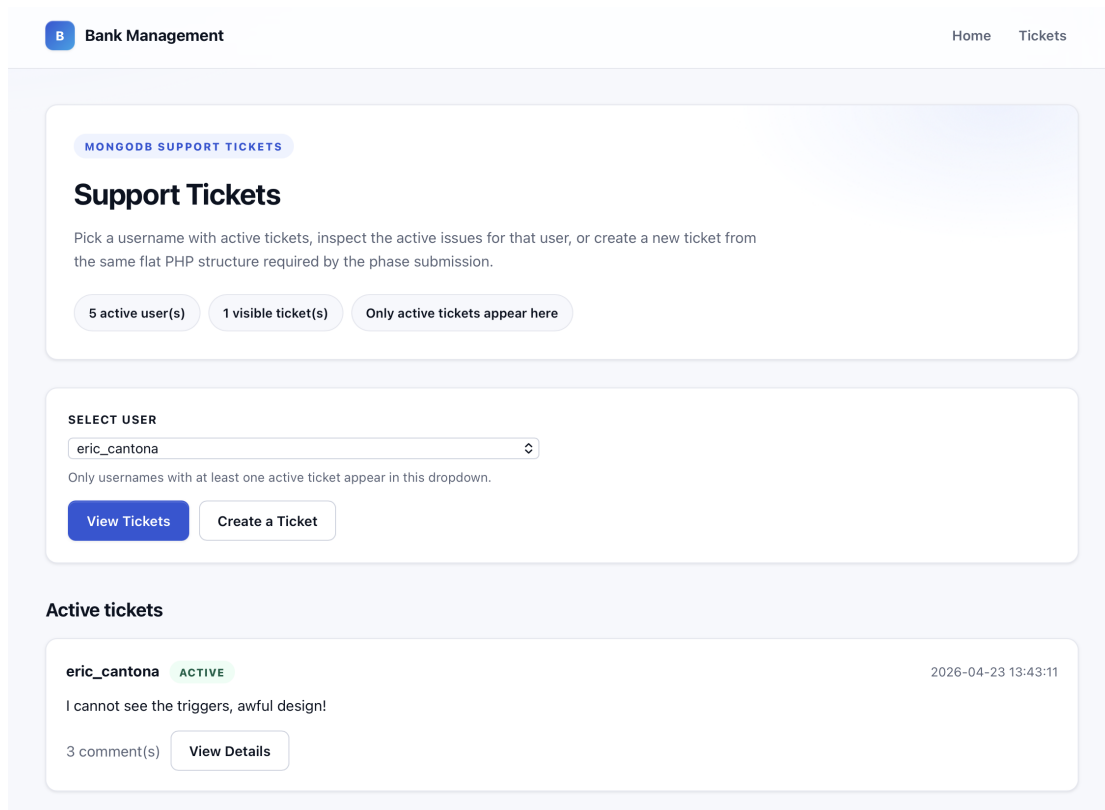



Figure 11: User ticket list page with the username filter and active tickets.

6.2 Ticket Creation Query

Users create tickets from `user/ticket.create.php`. The insertion is implemented with `MongoDB\Driver\BulkWrite`. New tickets are active by default and start with an empty comments array.

Listing 18: MongoDB insertion script for ticket creation.

```
$bulk = new MongoDB\Driver\BulkWrite();
$bulk->insert([
    'username' => trim((string) $username),
    'body' => trim((string) $body),
    'status' => true,
    'created_at' => date('Y-m-d H:i:s'),
    'comments' => [],
]);

ticket_manager()->executeBulkWrite(TICKETS_NAMESPACE, $bulk);
```

Bank Management Home Tickets

Create a Ticket

Open a new support ticket. The ticket will be stored in MongoDB with an active status and an empty comment list.

USERNAME
Naghi_mamouli

MESSAGE
How can I contact your UI designer?

Create Ticket Back to ticket list

< Back to dashboard

Figure 12: Ticket creation page used to submit a username and issue body.

Bank Management Home Tickets

Create a Ticket

Open a new support ticket. The ticket will be stored in MongoDB with an active status and an empty comment list.

TICKET CREATED
Ticket created successfully.

Create another ticket Back to ticket list

< Back to dashboard

Figure 13: Ticket creation confirmation page.

6.3 Ticket Detail Query

Ticket detail pages retrieve a single MongoDB document by `_id`. The string ID from the URL is converted into a `MongoDB\BSON\ObjectId` before querying.

Listing 19: MongoDB query for loading a single ticket by `ObjectId`.

```
$objectId = new MongoDB\BSON\ObjectId((string) $id);  
$query = new MongoDB\Driver\Query(['_id' => $objectId]);  
$results = ticket_manager()->executeQuery(TICKETS_NAMESPACE, $query)->toArray  
();
```

B

Bank Management

[Home](#)
[Tickets](#)

Ticket Details

Naghi_mamouli

ACTIVE

2026-05-07 15:19:26

How can I contact your UI designer?

Ticket metadata

- Username: Naghi_mamouli
- Created at: 2026-05-07 15:19:26
- Status: Active
- Comment count: 0

Comments

No comments yet.

Add a Comment

YOUR USERNAME

Naghi_mamouli

COMMENT

Figure 14: Ticket detail page showing metadata, status, body, and comments.

6.4 Comment Query

Both users and admins can add comments. The update pushes a new comment object into the `comments` array.

Listing 20: MongoDB update script for adding comments.

```
$bulk = new MongoDB\Driver\BulkWrite();
$bulk->update(
    ['_id' => $objectId],
    ['$push' => [
        'comments' => [
            'username' => trim((string) $username),
            'comment' => trim((string) $body),
            'created_at' => date('Y-m-d H:i:s'),
        ],
    ]],
    ['multi' => false, 'upsert' => false]
);

$result = ticket_manager()->executeBulkWrite(TICKETS_NAMESPACE, $bulk);
```

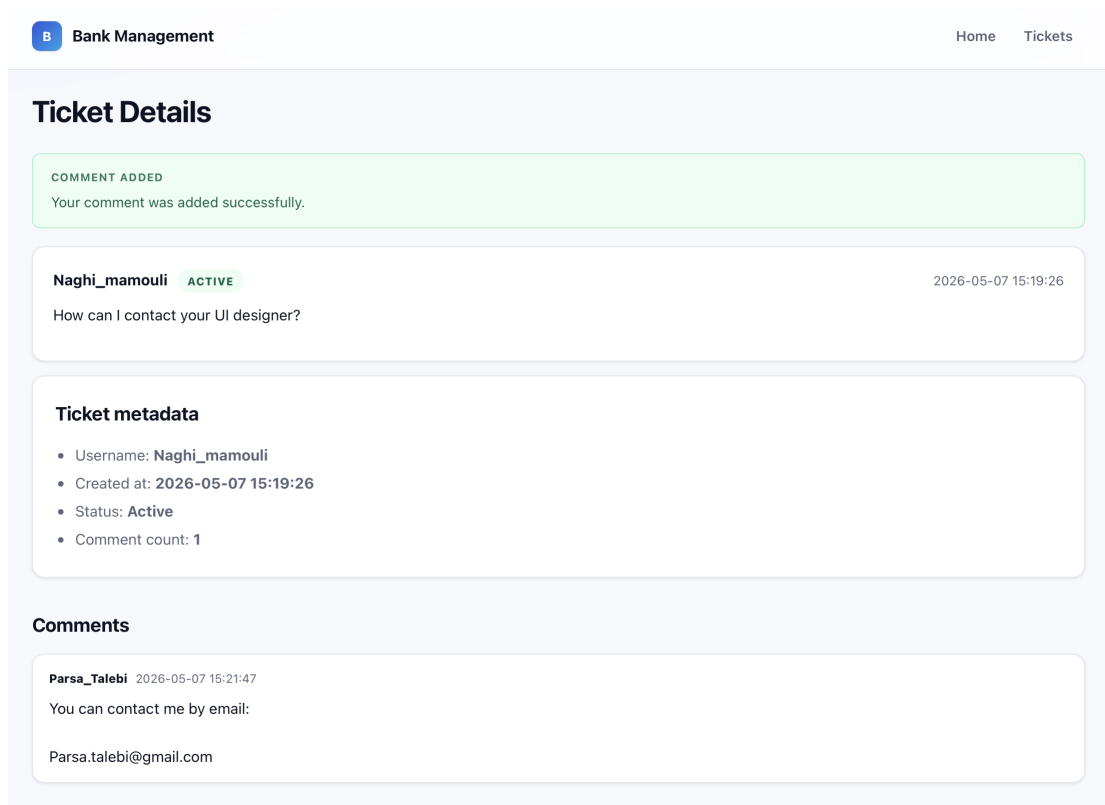


Figure 15: Ticket detail page after a comment was added.

6.5 Admin Resolve Query

The admin ticket detail page includes a resolve action. Resolving a ticket sets **status** to **false**. Since both user and admin list pages filter by **status => true**, the resolved ticket disappears from active ticket views.

Listing 21: MongoDB update script for resolving tickets.

```
$bulk = new MongoDB\Driver\BulkWrite();
$bulk->update(
    ['_id' => $objectId],
    ['$set' => ['status' => false]],
    ['multi' => false, 'upsert' => false]
);

$result = ticket_manager()->executeBulkWrite(TICKETS_NAMESPACE, $bulk);
```

A

Bank Management Admin

Active Tickets

User Site

ACTIVE TICKETS

6

Visible on the admin homepage right now

USERS AFFECTED

6

Distinct usernames with active tickets

COMMENT VOLUME

7

Existing comments on the active tickets

Ticket queue

Naghi_mamouli

ACTIVE

2026-05-07 15:19:26

How can I contact your UI designer?

1 comment(s)

View and Manage

Jamal_musiala

ACTIVE

2026-05-03 13:26:02

I have a problem!

1 comment(s)

View and Manage

Vincent_company

ACTIVE

2026-04-26 17:28:40

My team is winning UCL, but your site cannot even show correct triggers!

1 comment(s)

View and Manage

olivier_giroud

ACTIVE

2026-04-26 17:27:54

Figure 16: Admin active ticket list showing all active tickets from all users.

A

Bank Management Admin

Active Tickets

User Site

- Comments from this page are automatically stored with username **admin**.
- Resolving a ticket flips its **status** to **false**.
- Resolved tickets no longer appear on either homepage because both lists filter to active entries.

Comments

No comments yet.

Comment as admin

ADMIN COMMENT

Post Admin Comment

Back to active tickets

Resolve ticket

Use this when the issue is complete. The ticket will disappear from active user and admin views.

Mark Resolved

Cancel

Figure 17: Admin ticket detail page with comment and resolve actions.

A

Bank Management Admin

Active Tickets

User Site

TICKET RESOLVED

The ticket was marked resolved and removed from the active lists.

ACTIVE TICKETS

5

Visible on the admin homepage right now

USERS AFFECTED

5

Distinct usernames with active tickets

COMMENT VOLUME

7

Existing comments on the active tickets

Ticket queue

Naghi_mamouli

ACTIVE

2026-05-07 15:19:26

How can I contact your UI designer?

1 comment(s)

View and Manage

Jamal_musiala

ACTIVE

2026-05-03 13:26:02

I have a problem!

1 comment(s)

View and Manage

Vincent_company

ACTIVE

2026-04-26 17:28:40

My team is winning UCL, but your site cannot even show correct triggers!

Figure 18: Admin ticket list after resolving a ticket.

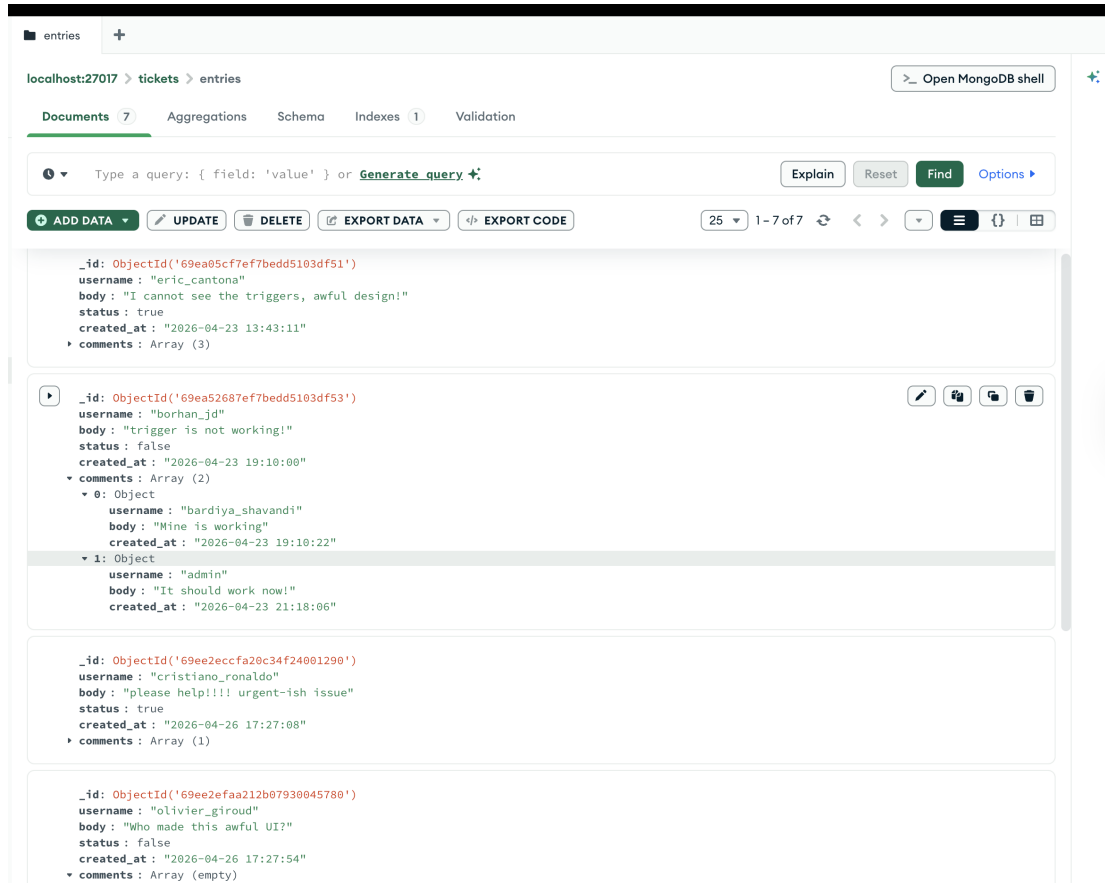


Figure 19: MongoDB Compass view of the ticket collection.

7 User Interface Design

The user interface was implemented using shared rendering functions in `common_ui.php`. The pages use a consistent navigation bar, card layout, form styles, output boxes, and table styles. The user interface uses a blue theme, while the admin interface uses a separate warm-colored theme to visually separate administrative actions from regular user actions.

The main dashboard includes three trigger cards, three procedure cards, and a support section. Each card contains the feature name, the responsible group member, and a button that opens the corresponding testing page.

8 Assumptions and Design Decisions

- The web application is intended to run locally with Apache, MySQL/MariaDB, PHP, and MongoDB available on the same machine.
- MySQL uses the database schema `bankmanagement`.
- MongoDB uses database `tickets` and collection `entries`.
- The support ticket workflow is intentionally lightweight and does not include password-based authentication. Username selection acts as a simple filter, as required by the phase specification.

- Resolved tickets remain stored in MongoDB but are hidden from active ticket list pages because the list queries filter by `status => true`.
- The helper trigger `trg_apply_transaction_to_balance` is not a user-facing required trigger; it is an internal consistency trigger used to keep account balances synchronized with transaction inserts.

9 Challenges and Fixes

- Apache initially returned a 403 error because it could not traverse the home directory. This was fixed by adjusting directory permissions with `chmod -R a+rX`.
- MariaDB initially rejected `CREATE PROCEDURE` statements with a `mysql.proc is wrong` error. Running `mysql_upgrade -u root` and restarting MySQL fixed the issue.
- Building the `ext-mongodb` PECL extension required pointing the build process to Homebrew OpenSSL using `LDFLAGS`, `CPPFLAGS`, and `PKG_CONFIG_PATH`, then running `sudo -E pecl install mongodb`.
- PHP 8's MySQLi exception behavior made trigger `SIGNAL` failures render as blank pages. Adding `mysqli_report(MYSQLI_REPORT_OFF)` made query failures return `false`, allowing the trigger message to be displayed in the page.
- Updating `SQLDump.sql` did not automatically update the running database. After SQL changes, procedures and triggers had to be manually dropped and recreated in phpMyAdmin's SQL tab.
- Some trigger pages originally removed or rolled back test rows automatically. Roll-back behavior was removed where persistence was needed for phpMyAdmin verification.

10 Conclusion

Phase 4 successfully integrates the Bank Management MySQL database into a PHP web application and adds MongoDB support tickets. The system demonstrates all three required triggers, all three required stored procedures, user-side ticket creation and tracking, and admin-side ticket management. The implementation also includes practical error handling for database-level validation failures and separates user and admin workflows through distinct pages.

A AI Usage Documentation

A.1 Borhan Javadian

ChatGPT was used in this conversation for Phase 4 report generation. The uploaded phase instructions, previous report format, SQL file, PHP scripts, project screenshots, and team AI usage notes were provided as context. The AI assistance was used to organize the report structure, convert the implementation details into LaTeX, summarize trigger/procedure behavior, and document MongoDB support ticket queries. The actual project implementation details were supplied from the submitted scripts and team notes.

A.2 Bardiya Shavandi

1. **Environment.** Apache returned 403 because it could not traverse `~/`; fixed with `chmod -R a+rX`. MariaDB rejected `CREATE PROCEDURE` with `mysql.proc is wrong` until `mysql_upgrade -u root` was run and MySQL was restarted. Building the `ext-mongodb` PECL extension required pointing it at Homebrew's OpenSSL via `LDFLAGS`, `CPPFLAGS`, and `PKG_CONFIG_PATH`, then running `sudo -E pecl install mongodb`.
2. **PHP-vs-MySQL exception behaviour.** PHP 8 surfaces MySQLi failures as exceptions, which made trigger `SIGNAL` errors render as blank pages. This was solved by adding `mysqli_report(MYSQLI_REPORT_OFF)` in `db.php` so query failures return `false` and the trigger message can be displayed.
3. **State persistence.** The trigger pages originally either auto-restored or auto-deleted the test row, which made changes invisible in phpMyAdmin. The rollback code was removed in Trigger 1 and Trigger 3 so successful operations persist.
4. **Missing live-database installs.** Updating `SQLDump.sql` does not refresh the running database. After every SQL change, such as the Procedure 2 logic update, Trigger 3 install, and the internal `trg_apply_transaction_to_balance` trigger, the object had to be manually dropped and recreated in phpMyAdmin's SQL tab.

A.3 Faraz Sahabi

1. **SQL Database Implementations.** AI assistance was used for implementing `trg_validate_transaction_amount`, a trigger that fires before insert on the `Transaction` table and validates that the transaction amount is strictly positive. AI assistance was also used for `GetTransactionCountByCustomer`, a stored procedure that receives a `CustomerID` and joins `Account` and `Transaction` to return the customer's transaction count.
2. **Web Interface Integration.** AI assistance was used in developing `trigger_3.php`, which tests Trigger 3 using an account dropdown and amount input, captures SQL errors from trigger failures, and displays the result. AI assistance was also used in developing `procedure_3.php`, which populates a customer dropdown and executes the stored procedure using MySQLi prepared statements. The dashboard was updated so the Trigger 3 and Procedure 3 cards link to the correct pages.

3. **Version Control.** AI assistance was used to resolve Git configuration issues by setting global `user.name` and `user.email`, and to commit the created SQL and PHP files to the local repository.

B Integration Notes

- Run Apache, MySQL/MariaDB, and MongoDB before opening the application.
- Import `SQLDump.sql` into MySQL/MariaDB to create the database, sample data, triggers, and stored procedures.
- Ensure the MongoDB PHP extension is enabled in `php.ini`.
- Open the user application through `localhost/user/index.php`.
- Open the admin application through `localhost/admin/index.php`.